Name : Soumya Pandey

Enrollment No. : 2303051240230

Roll_No. : 41

Div : 6B1

Topic : Mental Health Disease Prediction

Mental Health Disease Prediction

This system analyzes patterns from survey responses such as:

1. Work stress
2. Sleep habits
3. Social interaction
4. Family history
5. Workplace environment
6. Personal feelings

Machine Learning finds hidden relationships between these factors and mental health conditions.

In [25]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixD

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
```

In [26]:
```
df = pd.read_csv("/content/survey.csv")

print("First 5 Rows:")
df.head()
```

```
First 5 Rows:
```

Out[26]:

	Timestamp	Age	Gender	Country	state	self_employed	family_history	trea
0	2014-08-27 11:29:31	37	Female	United States	IL	NaN	No	
1	2014-08-27 11:29:37	44	M	United States	IN	NaN	No	
2	2014-08-27 11:29:44	32	Male	Canada	NaN	NaN	No	
3	2014-08-27 11:29:46	31	Male	United Kingdom	NaN	NaN	Yes	
4	2014-08-27 11:30:22	31	Male	United States	TX	NaN	No	

5 rows × 27 columns

In [27]:

```
print("\nDataset Info:")
df.info()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1259 entries, 0 to 1258
Data columns (total 27 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Timestamp                  1259 non-null   object
 1   Age                        1259 non-null   int64
 2   Gender                     1259 non-null   object
 3   Country                    1259 non-null   object
 4   state                      744 non-null    object
 5   self_employed              1241 non-null   object
 6   family_history             1259 non-null   object
 7   treatment                  1259 non-null   object
 8   work_interfere             995 non-null    object
 9   no_employees               1259 non-null   object
 10  remote_work                1259 non-null   object
 11  tech_company               1259 non-null   object
 12  benefits                   1259 non-null   object
 13  care_options               1259 non-null   object
 14  wellness_program           1259 non-null   object
 15  seek_help                  1259 non-null   object
 16  anonymity                  1259 non-null   object
 17  leave                      1259 non-null   object
 18  mental_health_consequence  1259 non-null   object
 19  phys_health_consequence    1259 non-null   object
 20  coworkers                  1259 non-null   object
 21  supervisor                 1259 non-null   object
 22  mental_health_interview    1259 non-null   object
 23  phys_health_interview      1259 non-null   object
 24  mental_vs_physical         1259 non-null   object
 25  obs_consequence            1259 non-null   object
 26  comments                   164 non-null    object
dtypes: int64(1), object(26)
memory usage: 265.7+ KB
```

In [28]:

```python
# Fill missing values
df = df.fillna(method='ffill')

print("Missing values handled")
```

```
Missing values handled
```

```
/tmp/ipykernel_15320/3859885903.py:2: FutureWarning: DataFrame.fillna with 'met
hod' is deprecated and will raise in a future version. Use obj.ffill() or obj.b
fill() instead.
  df = df.fillna(method='ffill')
```

In [29]:

```python
le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

print("Categorical data converted")
```

```
Categorical data converted
```

In [30]:
```
#target variable
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

In [31]:
```
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale the data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Update X_train and X_test to be scaled versions for model training
X_train = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test = pd.DataFrame(X_test_scaled, columns=X_test.columns)
```

In [32]:
```
models = {
    "Logistic Regression": LogisticRegression(max_iter=5000), # Increased max_
    "Random Forest": RandomForestClassifier(n_estimators=100),
    "Decision Tree": DecisionTreeClassifier(),
    "KNN": KNeighborsClassifier()
}

results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred) * 100
    results.append((name, acc))

    print(f"{name} Accuracy: {acc:.2f}%")
```

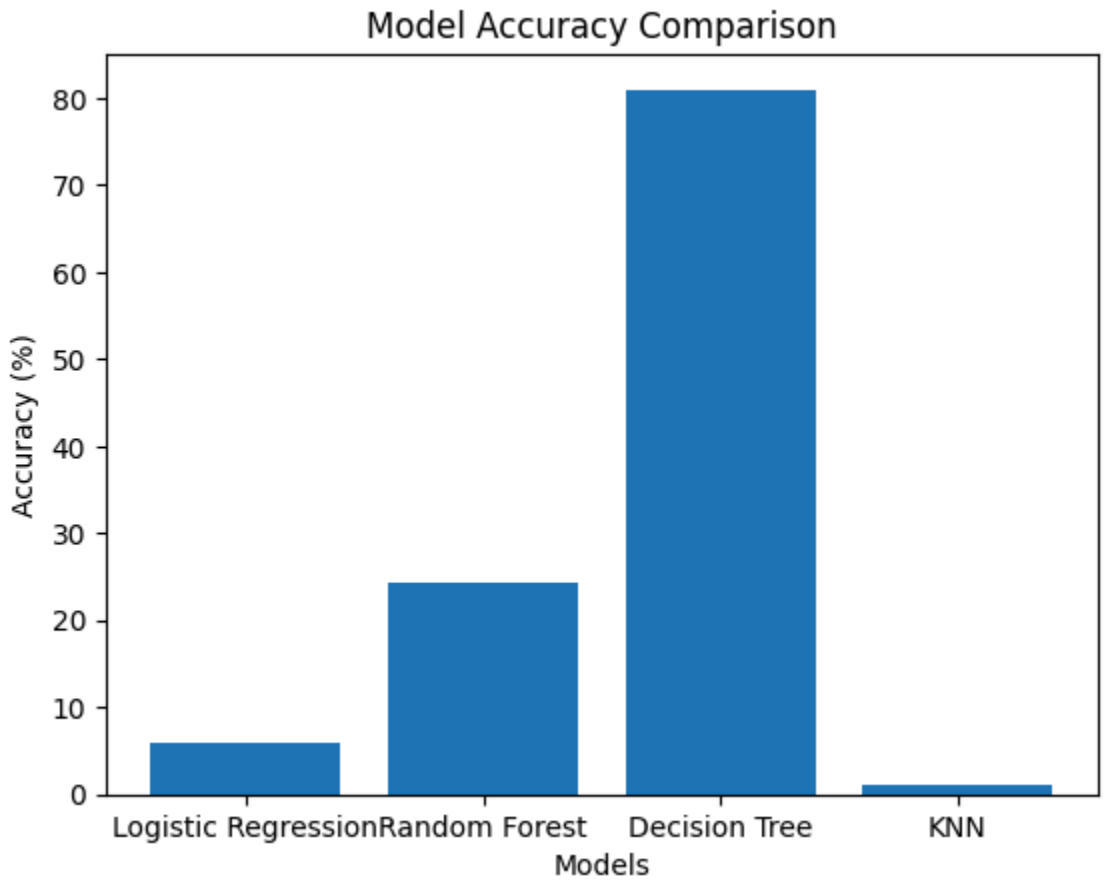
```
Logistic Regression Accuracy: 5.95%
Random Forest Accuracy: 24.21%
Decision Tree Accuracy: 80.95%
KNN Accuracy: 1.19%
```

Data Visuals

Bar Graph

In [33]:
```
results_df = pd.DataFrame(results, columns=["Model", "Accuracy"])

plt.figure()
```

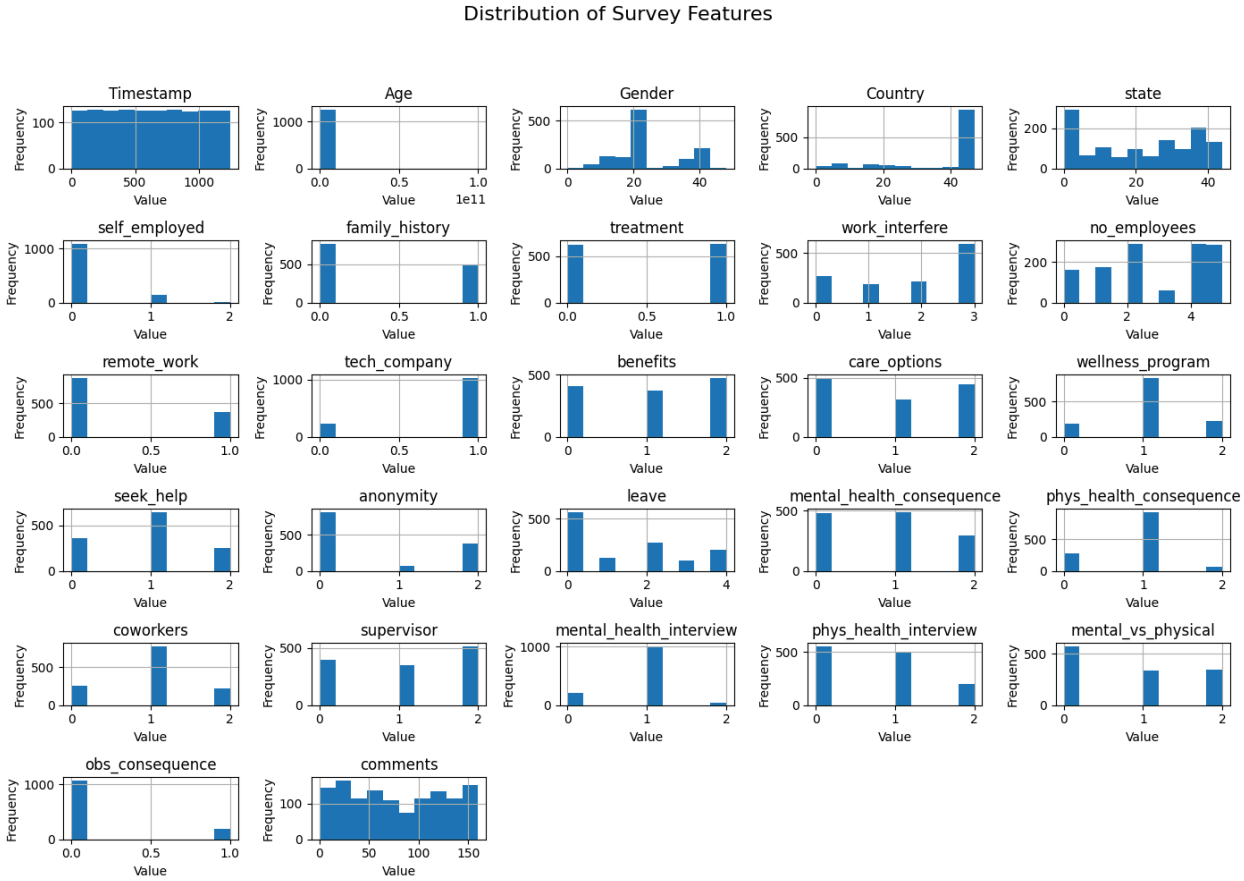
```
plt.bar(results_df["Model"], results_df["Accuracy"])
plt.title("Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy (%)")
plt.show()
```



Histogram

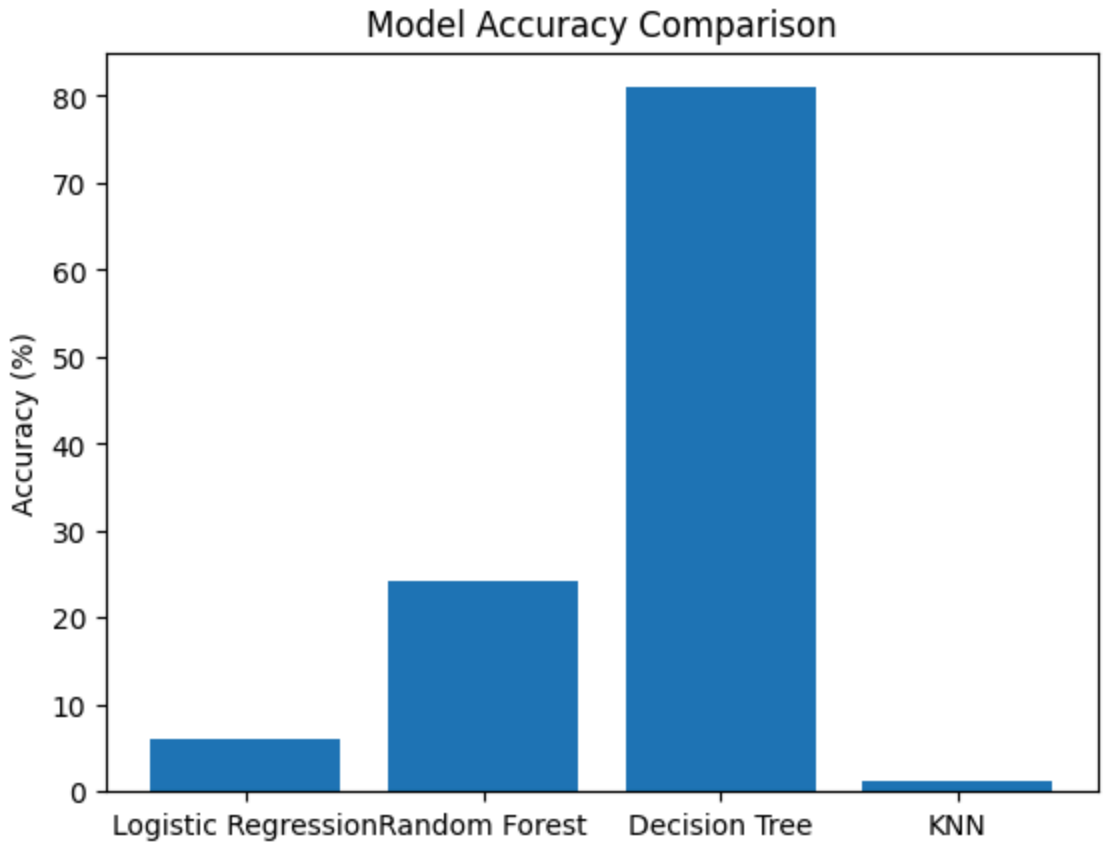
In [34]:
```
plt.figure(figsize=(14,10))
hist_plots = df.hist(figsize=(14,10))
plt.suptitle("Distribution of Survey Features", y=1.02, fontsize=16) # Adjust

# Add axis labels to each subplot
for row_plots in hist_plots:
    for ax in row_plots:
        ax.set_xlabel("Value")
        ax.set_ylabel("Frequency")

plt.tight_layout(rect=[0, 0.03, 1, 0.98]) # Adjust layout to prevent overlap w
plt.show()
```

```
<Figure size 1400x1000 with 0 Axes>
```



Accuracy Comparison Graph (Models)

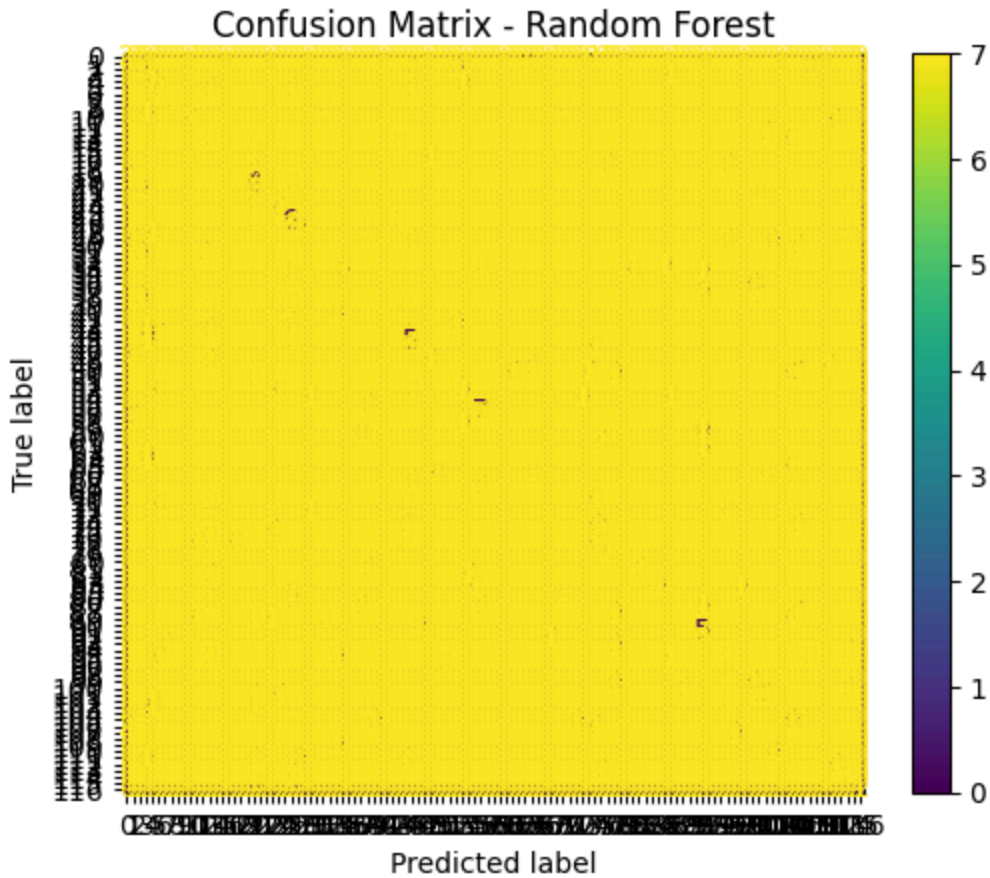
In [35]:
```
plt.figure()
plt.bar(results_df["Model"], results_df["Accuracy"])
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy (%)")
plt.show()
```



This Shows that the Decision Tree works better for our project

Confusion Matrix

In [36]:
```
best_model = RandomForestClassifier()
best_model.fit(X_train, y_train)

y_pred = best_model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(cm)
disp.plot()
plt.title("Confusion Matrix - Random Forest")
plt.show()
```



In [37]:
```
# Train Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

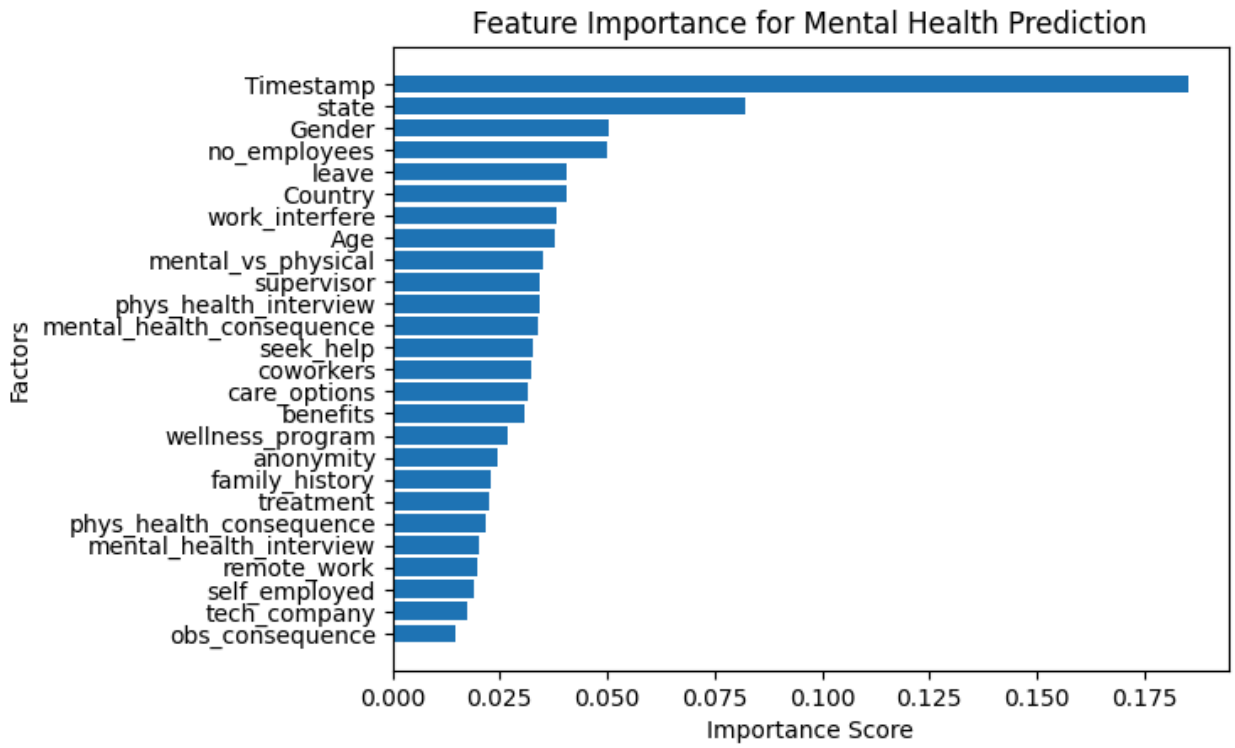
Out[37]:
```
▾ RandomForestClassifier
RandomForestClassifier(random_state=42)
```

In [38]:
```
# Get importance of each feature
importances = rf.feature_importances_

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importances
})

# Sort by importance
feature_importance = feature_importance.sort_values(by='Importance', ascending

print(feature_importance)
```

```
                      Feature  Importance
0                   Timestamp    0.185266
4                       state    0.082110
2                      Gender    0.050282
9                no_employees    0.050106
17                      leave    0.040661
3                     Country    0.040439
8              work_interfere    0.038303
1                         Age    0.037798
24         mental_vs_physical    0.035048
21                 supervisor    0.034495
23      phys_health_interview    0.034362
18  mental_health_consequence    0.033727
15                  seek_help    0.032890
20                  coworkers    0.032375
13               care_options    0.031731
12                   benefits    0.030665
14           wellness_program    0.026708
16                  anonymity    0.024435
6              family_history    0.022763
7                   treatment    0.022570
19    phys_health_consequence    0.021908
22    mental_health_interview    0.020182
10                remote_work    0.019911
5               self_employed    0.018917
11               tech_company    0.017624
25            obs_consequence    0.014722
```

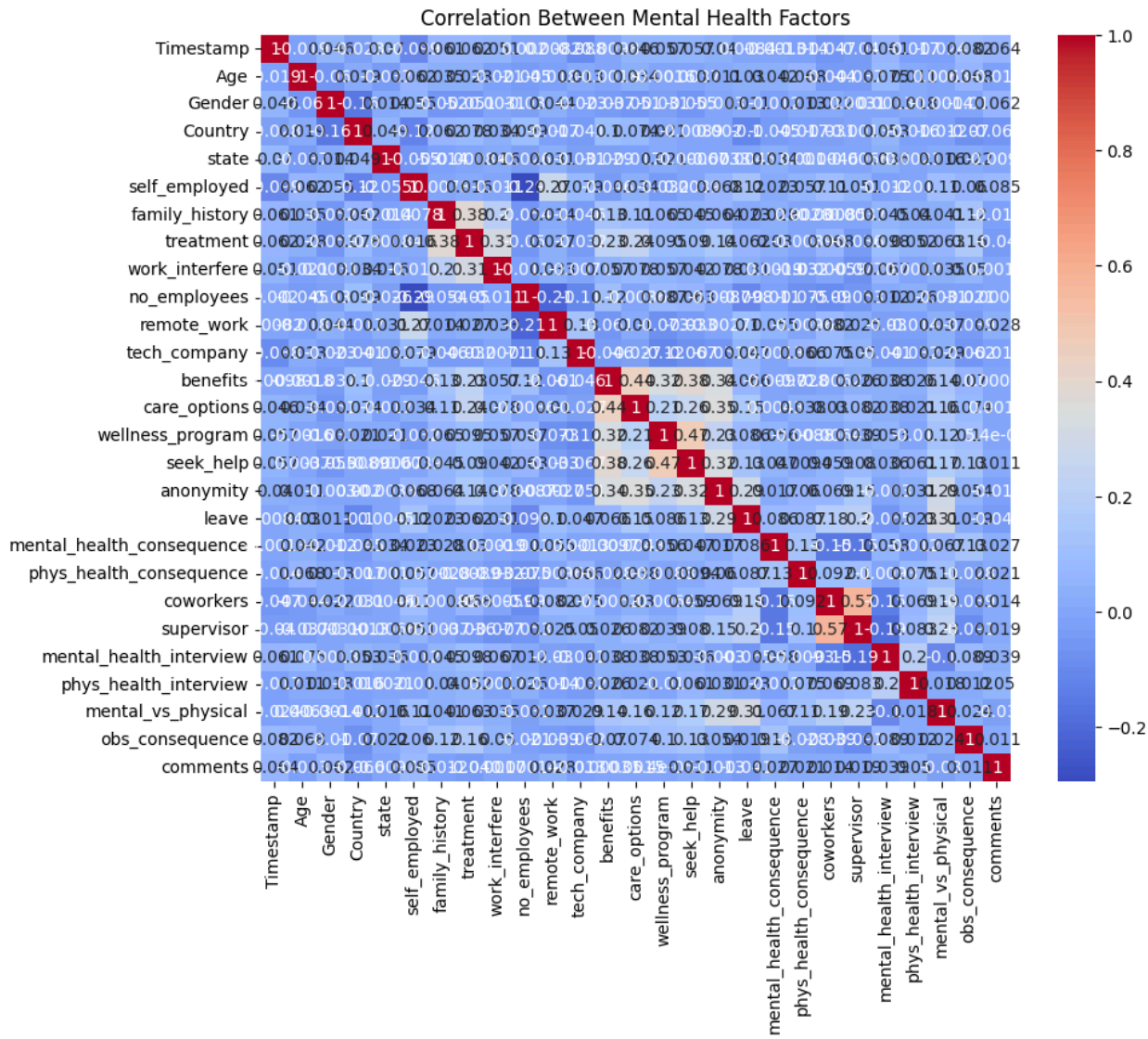
In [39]:

```python
import matplotlib.pyplot as plt

plt.figure()
plt.barh(feature_importance['Feature'], feature_importance['Importance'])
plt.title("Feature Importance for Mental Health Prediction")
plt.xlabel("Importance Score")
plt.ylabel("Factors")
plt.gca().invert_yaxis()
plt.show()
```



Correlation Matrix

In [40]:
```
import seaborn as sns

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Between Mental Health Factors")
plt.show()
```



In [41]:
```
def mental_state(score):
    if score == 0:
        return "😊 Happy"
    elif score == 1:
        return "😐 Sad"
    elif score == 2:
        return "😟 Stressed"
    else:
        return "😞 Depressed"
```

In [42]:
```
#Example

new_person = X_test.iloc[0:1]

prediction = best_model.predict(new_person)[0]

print("Predicted Mental Health State:", mental_state(prediction))
```

```
Predicted Mental Health State: 😞 Depressed
```

How It Shows Mental Health Prediction

The system:

1. Collects user survey responses
2. Converts answers into numerical data
3. Applies trained ML model
4. Classifies mental health condition
5. Displays emotional state

Benefits:

1. Social Benefits

✔ Early detection of mental health issues

✔ Helps in preventive care

✔ Reduces stigma around mental illness

✔ Supports healthcare professionals

2. Academic Benefits

✔ Demonstrates supervised learning

✔ Shows real-world application of AI

✔ Useful for healthcare analytics

3. Practical Benefits

✔ Can be used in workplaces

✔ Useful for student counseling systems

✔ Can evolve into a mobile/web app

Conclusion:

This project developed an AI-based Mental Health Prediction System using

supervised machine learning techniques.

1. The system analyzes survey data to identify patterns related to an individual’s psychological well-being.
2. Multiple algorithms were implemented and compared to determine the most accurate prediction model.
3. The model predicts mental health conditions and classifies individuals into states such as Happy, Sad, Stressed, or Depressed.
4. The system demonstrates the effectiveness of data-driven approaches for early detection of mental health issues.
5. It can assist organizations, educational institutions, and healthcare providers in monitoring mental well-being.

Overall, the project highlights the potential of Artificial Intelligence and Machine Learning in improving mental health awareness and support systems.